

Media Fullscreen Viewer Skill

1. Skill 定位

本 Skill 用于规范跨平台媒体应用中的全屏图片和视频查看功能。

支持平台：

- iOS
- Android
- Linux

技术架构：



核心原则：

图片和视频使用统一的 Media Viewer 架构。

图片和视频共享媒体操作能力。

播放控制属于 Video Viewer，不属于通用 Media Viewer。

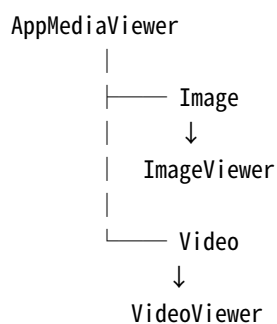
添加到相册和添加标签属于通用 Media Action。

2. Media Viewer 架构

统一入口：

```
AppMediaViewer(  
    media: media,  
);
```

根据媒体类型自动选择：



业务层不得直接判断：

```
if(media.type == MediaType.image) {  
    ...  
}  
  
if(media.type == MediaType.video) {  
    ...  
}
```

媒体类型判断应该封装在：

```
AppMediaViewer
```

内部。

3. 全屏查看器设计原则

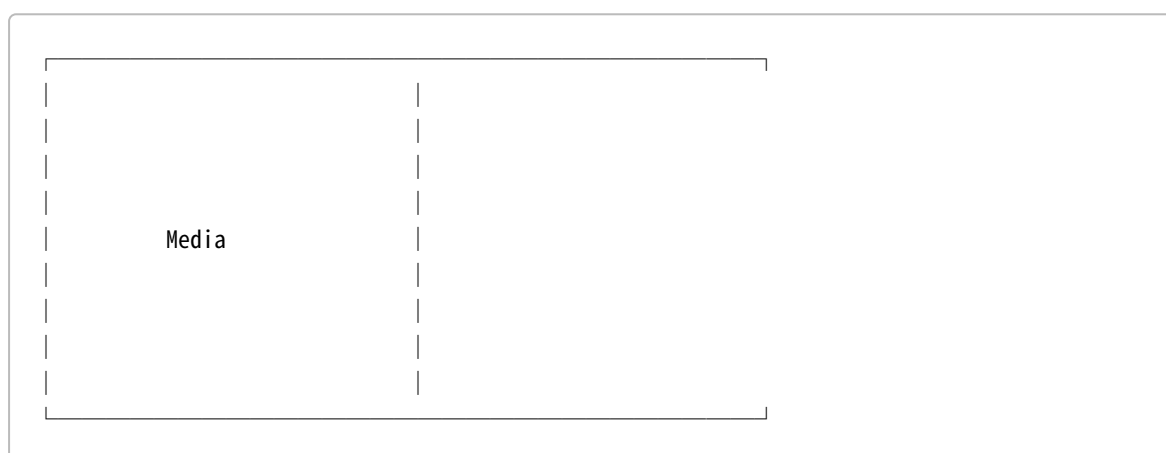
全屏查看器必须采用：

Immersive Media Viewing

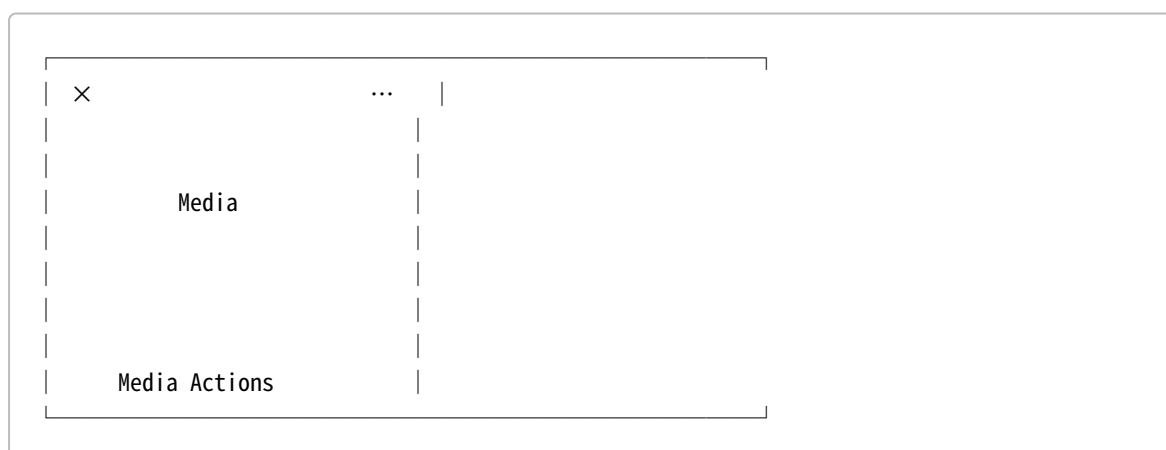
即：

- 默认最大化媒体显示区域
- 默认隐藏非必要 UI
- 点击屏幕显示控制 UI
- 点击空白区域隐藏控制 UI
- 支持手势
- 支持横竖屏
- 支持安全区域
- 支持系统状态栏适配

默认状态：



用户点击屏幕：



UI 控件必须支持自动隐藏。

4. Image Viewer

4.1 功能

图片查看器必须支持：

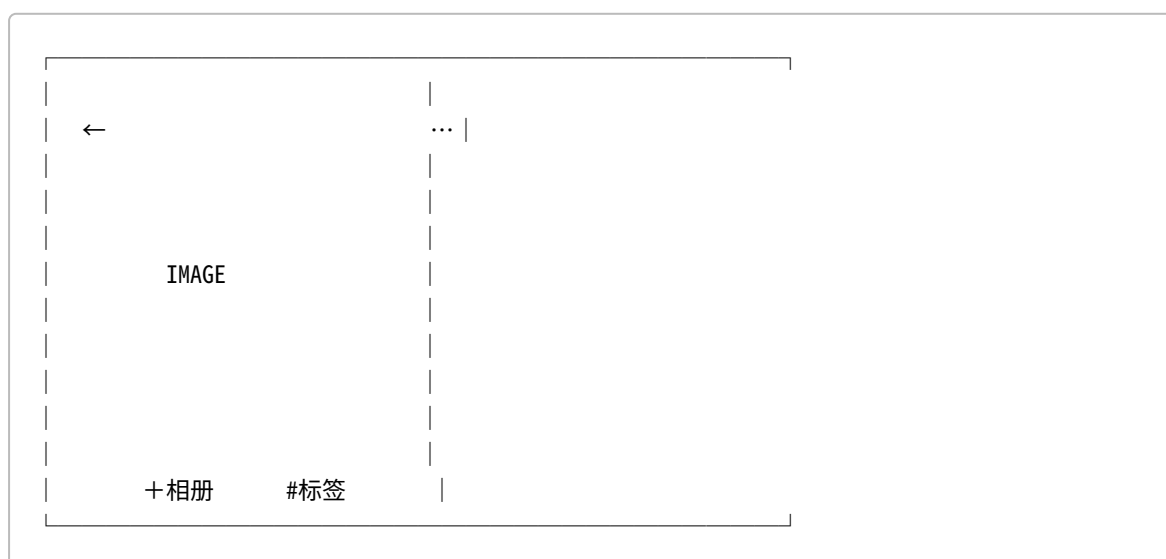
- 全屏查看
- 双指缩放
- 双指缩小
- 拖拽
- 双击缩放
- 双击恢复
- 左右滑动切换图片
- 点击显示/隐藏 UI
- 返回
- 添加到相册
- 添加标签

可选：

- 旋转
- 分享
- 收藏
- 删除
- 图片信息

5. 图片布局

默认：



顶部：

Back
More

底部：

Add To Album
Add Tag

如果屏幕空间不足：

...
|
|—— 添加到相册
|—— 添加标签
|—— 分享
|—— 收藏
|—— 图片信息
|—— 删除

6. 图片手势

必须支持：

单击
↓
显示 / 隐藏 UI

双击
↓
缩放 / 恢复

双指
↓
缩放

拖动
↓
移动图片

左右滑动
↓
切换媒体

手势优先级：

Zoom Gesture



Pan Gesture



Page Swipe

当图片处于放大状态时：

左右拖动

优先移动图片。

当图片处于默认比例时：

左右滑动

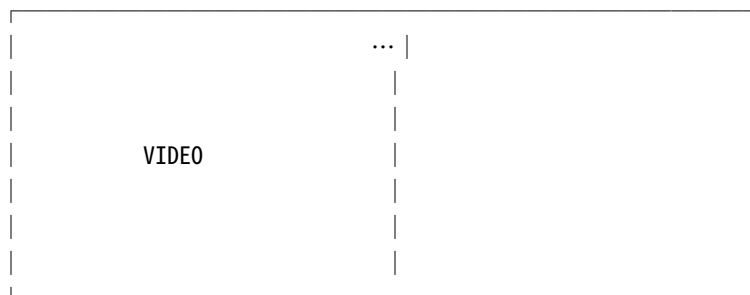
切换下一张图片。

7. Video Viewer

视频查看器采用：

Immersive Video Player

默认：



点击屏幕：





8. 视频播放控制

Video Viewer 必须支持：

- 播放
- 暂停
- 播放进度
- 拖动进度
- 音量
- 静音
- 全屏
- 倍速
- 添加到相册
- 添加标签

可选：

- 下一条
- 上一条
- 循环播放
- 字幕
- 音轨

播放控制 UI 必须自动隐藏。

推荐：

用户无操作



2~3 秒



自动隐藏控制栏

用户点击：

显示控制栏

9. Video Action 设计

播放控制和媒体管理操作必须分离。

播放控制：

```
Play  
Pause  
Seek  
Volume  
Speed
```

媒体操作：

```
Add To Album  
Add Tag  
Share  
Favorite  
Delete  
Info
```

媒体操作放在：

```
More Menu
```

或者：

```
底部 Action Bar
```

禁止把大量管理操作长期显示在视频播放区域。

10. AppMediaActions

图片和视频共享：

```
AppMediaActions(  
    media: media,  
);
```

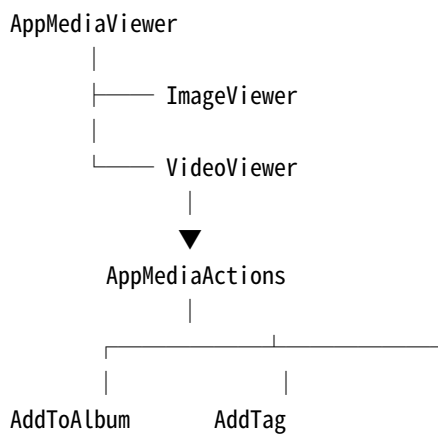
标准操作：

Add To Album
Add Tag

可选操作:

Share
Favorite
Info
Delete

架构:



11. Add To Album

11.1 功能

用户可以:

- 查看已有相册
- 选择一个相册
- 同时选择多个相册
- 取消已有相册关系
- 创建新相册
- 确认添加

媒体与相册关系必须支持多对多。



└── Album B
└── Album C

一个媒体可以属于多个相册。

12. Add To Album UI

推荐使用：

AppSheet

iOS：

Bottom Sheet / Sheet

Android：

Material Bottom Sheet

Linux：

Dialog / Sheet

UI：

添加到相册	
☒ 风景	
☐ 旅行	
☐ 日本	
☐ 收藏	
+ 新建相册	
取消	完成

如果已有相册关系：



如果没有：



13. 多选模式

支持：

Media
↓
Select Albums
↓
Multiple Selection
↓
Confirm

用户可以同时选择：

- ☒ 旅行
- ☒ 日本
- ☒ 风景

确认后：

Media
├── 旅行
├── 日本
└── 风景

14. 新建相册

点击：

+ 新建相册

打开：

AppDialog

UI:

新建相册

相册名称

日本旅行

取消 创建

创建完成后：

自动选择新相册

15. Add Tag

15.1 功能

用户可以：

- 查看已有标签
- 添加已有标签
- 创建新标签
- 删除标签
- 多选标签
- 搜索标签

媒体与标签关系必须支持多对多。

Media

|
|—— Tag: 旅行
|—— Tag: 日本
|—— Tag: 风景

16. Add Tag UI

推荐使用：

AppSheet

UI：

添加标签

[旅行 ×] [日本 ×]

搜索或输入标签

推荐

#风景 #建筑 #自然 #旅行

+ 创建新标签

取消 完成

17. Tag Interaction

点击推荐标签：

#旅行

自动添加：

[旅行 ×]

输入：

日本

显示：

已有标签

日本

创建新标签

+ 日本

如果不存在：

+ 创建「日本」

18. Tag Model

Tag 必须是独立实体。

Tag

|— id
|— name
|— createdAt
|— updatedAt

媒体关系：

Media

|



MediaTag

|



Tag

数据库：

media

tags

media_tags

关系：

Media

|

| N



MediaTag



N

Tag

即：

Many-to-Many

19. Album Model

Album：

Album

id
name
coverMediaId
createdAt
updatedAt

媒体关系：

Media



MediaAlbum



Album

数据库：

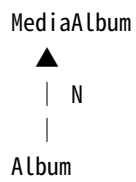
media
albums
media_albums

关系：

Media



N



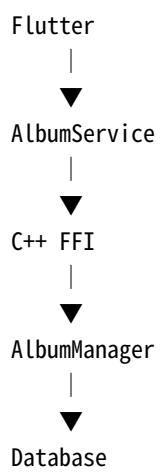
同样采用：

Many-to-Many

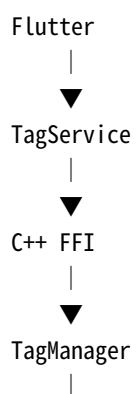
20. C++ Core

Flutter 不直接操作数据库。

架构：



Tag:



21. Media Action API

推荐统一：

MediaService

提供：

```
addToAlbum(  
    mediaId,  
    albumId  
)  
  
removeFromAlbum(  
    mediaId,  
    albumId  
)  
  
getAlbumsForMedia(  
    mediaId  
)  
  
addTag(  
    mediaId,  
    tagId  
)  
  
removeTag(  
    mediaId,  
    tagId  
)  
  
getTagsForMedia(  
    mediaId  
)
```

Flutter 不直接操作数据库。

22. Media Viewer 数据流

图片：

```
User
↓
Tap Image
↓
AppMediaViewer
↓
ImageViewer
↓
AppMediaActions
↓
AddToAlbum / AddTag
↓
Service
↓
C++ Core
↓
Database
```

视频:

```
User
↓
Tap Video
↓
AppMediaViewer
↓
VideoViewer
↓
Video Controls
+
AppMediaActions
↓
Service
↓
C++ Core
↓
Database
```

23. Native Platform Rules

系统能力优先使用平台官方 API。

iOS:

Photos Framework
Files
Share Sheet
System Picker

Android:

MediaStore
Photo Picker
SAF
Sharesheet

Linux:

File System
Desktop Integration
GTK / Dbus when necessary

禁止重新实现:

系统文件选择器
系统分享
系统权限 UI
系统照片选择器

24. UI Platform Rules

UI 必须根据平台适配。

iOS:

Apple HIG
Liquid Glass
Sheet
Popover
Context Menu

Android:

Material 3
Bottom Sheet

Dialog
Menu

Linux:

Desktop Layout
Sidebar
Dialog
Context Menu

Flutter Design System 只负责统一语义。

平台 Adapter 负责视觉实现。

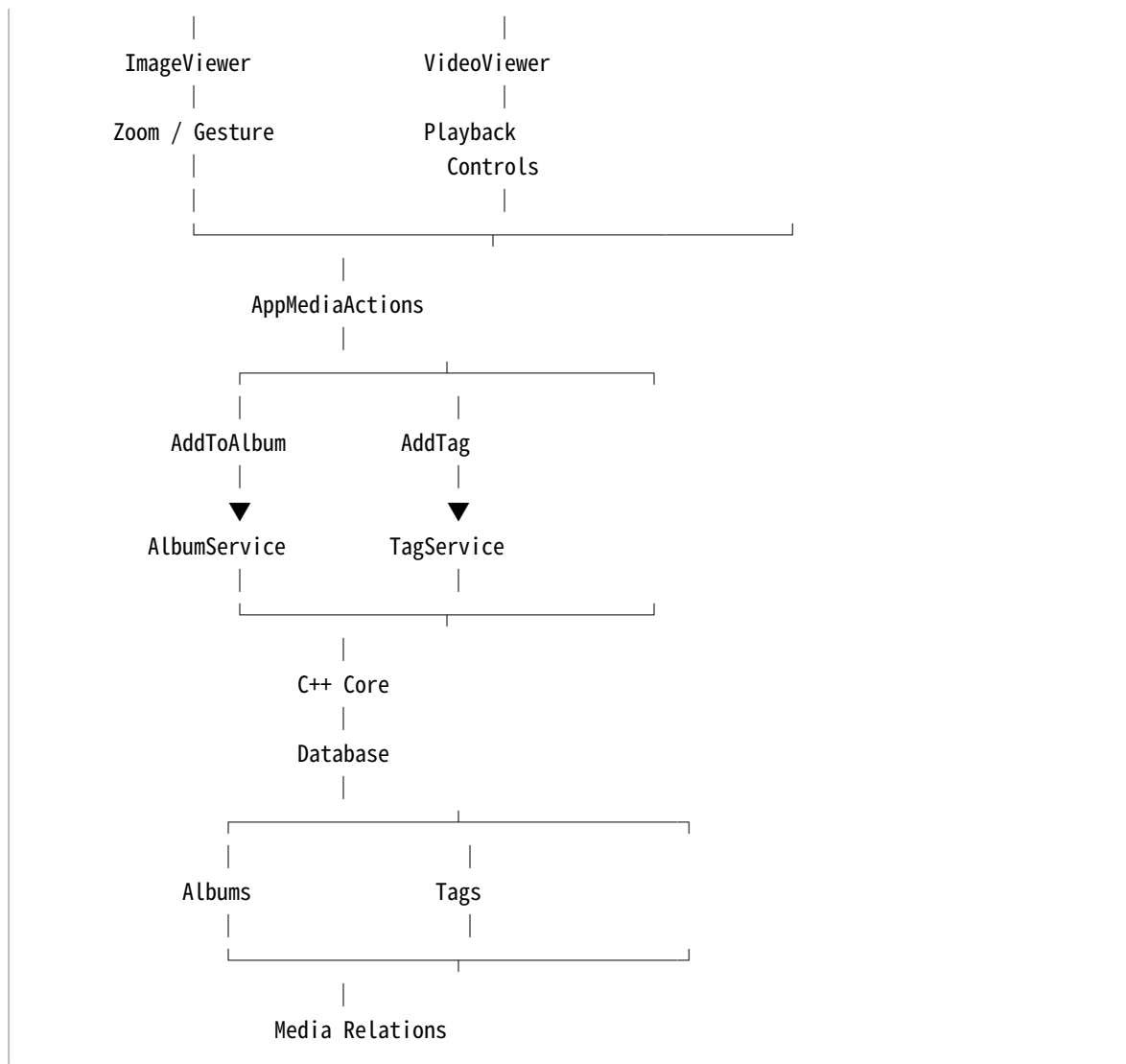
25. AI Coding Agent 行为

实现全屏媒体功能时：

1. 必须优先复用现有 Media Model。
2. 必须优先复用现有 Design System。
3. 图片和视频必须共用 Media Viewer 入口。
4. 图片和视频必须共用 Media Action。
5. 添加到相册必须支持多对多。
6. 添加标签必须支持多对多。
7. UI 不得直接操作数据库。
8. Flutter 不得直接实现核心数据库逻辑。
9. C++ 不得依赖 Flutter UI。
10. 平台差异不得散落在业务页面。
11. iOS、Android、Linux 使用各自官方设计语言。
12. 系统能力优先使用官方 API。
13. 任何系统 UI 不得自行模拟。
14. 视频控制和媒体管理操作必须分离。
15. 全屏查看必须优先保证媒体内容展示区域。
16. 操作 UI 默认自动隐藏。
17. 所有操作必须考虑触摸、鼠标和键盘输入。
18. 桌面端必须支持 Escape 返回。
19. 移动端必须遵循系统返回行为。
20. 不能因为跨平台而牺牲平台体验。

26. 最终 UI 架构

AppMediaViewer



最终目标：

用户看到的是沉浸式、符合 iOS / Android / Linux 官方设计语言的全屏媒体体验；Flutter 负责完整的界面和交互；C++ 负责媒体、相册、标签和数据库核心能力；Native 负责系统级能力。

图片和视频共用 MediaViewer 与 MediaActions；视频额外拥有播放控制；相册和标签都是独立实体，并与 Media 建立多对多关系。